

MLFQ: Multilevel Feedback Queue

Implemented rules and examples for Scheduling Studio. These describe the current visualizer, including its deterministic tie and boundary ordering.

The simulator uses one CPU, discrete ticks, and CPU-only processes. The normal UI has three queues: Q0 (highest), Q1, and Q2 (lowest). The engine supports any nonempty queue configuration.

There is no I/O, blocking, or context-switch cost in the normal UI. A snapshot at time t shows the state after decisions at t , before execution during $[t, t + 1)$. Completion from that interval is handled at $t + 1$.

Each queue has two independent positive whole-number settings:

- **Quantum (time slice):** maximum CPU ticks in a turn before rotating within that queue.
- **Allotment:** cumulative CPU ticks at that priority before demotion.

Both settings default to $Q0 = 2$, $Q1 = 4$, $Q2 = 8$. The default boost interval is 10 ticks. Engine callers that omit `mlfqAllotments` use their quantum values as allotments for compatibility. Omitting the engine's boost interval disables boosts.

Service is the total CPU work a process needs to finish. Cards label remaining work explicitly, for example `6 service left`. MLFQ cards also show `Q: used/total` · `A: used/total`; these are the current turn's quantum and the cumulative allotment at the current priority. Waiting time consumes neither budget nor service. The settings explain both budgets, and the CPU panel and metrics table show their remaining values separately.

The **Quantum vs allotment example** in the MLFQ settings loads A and B with 10 service ticks each, Q0 quantum 1 and allotment 4, and boost interval 100. They alternate in Q0 for four one-tick turns each, then enter Q1 with six service ticks remaining. A demotes at $t=7$; B demotes at $t=8$. Loading this example resets playback and opens metrics; the usual defaults are unchanged.

1. Highest priority runs first

Dispatch the head of the highest-priority nonempty queue. Higher-priority ready work preempts lower-priority running work. Return the preempted process to the front of its current queue, preserving its priority, partial quantum usage, and accumulated allotment. There is no boost protection.

Example: A runs in Q1 with `Q: 1/3` and `A: 4/7`; C waits in Q1. B arrives in Q0. A returns ahead of C in Q1, and B runs. When A resumes, it still has two quantum ticks and three allotment ticks remaining.

2. Same priority uses Round Robin

A same-priority arrival does not interrupt the runner. When its quantum expires and it has allotment remaining, append it to the back of the same queue. Reset quantum usage only.

Example: Q0 has quantum 1 and allotment 4. A runs for one tick while B waits. A returns behind B with `Q: 0/1` and `A: 1/4`. B runs next.

3. Arrivals join Q0

New processes enter the back of Q0 with both counters zero. Simultaneous arrivals retain input order.

Example: Input order C, A, B with identical arrivals gives Q0 order C, A, B before dispatch. IDs are not sorted alphabetically.

4. Execution, completion, and demotion

Each CPU tick reduces total remaining CPU service by one and increases quantum and allotment usage by one. Remaining CPU service is the work still needed to finish, not the time remaining in a turn.

At the next boundary, completion is handled first. A completed process leaves, even if a quantum, allotment, or boost expires simultaneously.

For an unfinished process, allotment exhaustion takes precedence over quantum expiry. Reset both counters and append it to the next lower queue. At the lowest queue, append it to that queue's tail with fresh counters. Allotment exhaustion may interrupt a partly used quantum.

Example: quantum 1 and allotment 4 allow four one-tick turns before demotion, assuming no intervening boost or completion. Quantum 2 and allotment 3 allow a two-tick turn followed by a one-tick turn before demotion.

Completion example: A has one service tick left with Q: 1/2 and A: 1/2. After one tick, A finishes rather than being demoted or boosted, even if a boost is due.

Bottom-queue example: A exhausts its Q2 allotment while B waits in Q2. A joins behind B with both counters zero and stays at Q2. If only its quantum expires and allotment remains, it instead retains the used allotment.

5. Waiting-process boost order

Boost at positive multiples of the interval, never at $t=0$. Collect waiting queues from highest to lowest priority, preserving head-to-tail order within each queue. Move these processes to Q0 and reset both usage counters.

Example: Waiting queues Q0 [C], Q1 [B, D], Q2 [E] become Q0 [C, B, D, E]. C also gets fresh counters even though it was already in Q0. Finished processes and future arrivals are not part of this collection.

6. Boost the ongoing runner too

If a process still has an ongoing turn after completion and expiry/yield handling, preempt it and append it after all waiting processes in Q0. Reset its quantum and allotment usage without changing remaining CPU service. Dispatch normally after same-time arrivals. With no other work, the same process can immediately run again.

Example: CPU A, Q1 B C, Q2 D E becomes Q0 B C D E A before dispatch; B runs next.

7. Expiry before boost

If an allotment expires at the boost boundary, demote and enqueue the process first. If only its quantum expires, rotate and enqueue it first. An early yield is also enqueued first. Then boost the queues.

Such a process participates in its queue's position in the flattened order; it is not necessarily last. For example, a runner demoted from Q0 to Q1 is boosted before existing Q2 work.

Example: A exhausts its Q0 allotment at a boost boundary; waiting queues are Q0 [B], Q1 [C], Q2 [D]. First enqueue A to make Q1 [C, A]. Then boost to Q0 [B, C, A, D], with fresh counters, before dispatch.

8. Arrivals and expiry without a boost

Admit arrivals before enqueueing a process whose quantum/allotment expired or that yielded. Check higher-priority preemption and dispatch afterward.

Example: A's Q0 quantum expires with allotment remaining. B already waits in Q0 and C arrives at the same time. Admit C, then rotate A: Q0 becomes [B, C, A]. B runs next.

9. Complete boundary order

1. Remove a completed runner.
2. Detect allotment exhaustion, quantum expiry, or early yield, in that order.
3. On boost boundaries, enqueue expired/yielded work, boost waiting work, and append any ongoing runner with fresh budgets.
4. Admit new arrivals.
5. On ordinary boundaries, enqueue expired/yielded work.
6. Apply ordinary higher-priority preemption, preserving both counters.
7. Dispatch if the CPU is free.
8. Record the boundary state, then execute the next CPU tick.

Example: A still has an ongoing turn in Q1 at a boost boundary, C waits in Q1, D waits in Q2, and E arrives at this boundary. Boosting gives Q0 [C, D, A]; admitting E gives [C, D, A, E]. C runs next.

10. Idle periods and completion of the workload

If no process is running or ready, record idle ticks until the next arrival. Boosts during an empty interval do not admit future work early. Stop after all processes finish. A lone process can be demoted, rotated, or boosted and immediately redispached at the same boundary, with no lost execution tick.

Example: A arrives at 3 with one service tick and the boost interval is 1. The CPU remains idle during 0–3; A enters Q0 at 3, runs during 3–4, and finishes at 4.

Worked example: four turns before demotion

Use the loadable example: input order A, B; both arrive at zero and need ten service ticks. Quanta are [1, 4, 8], allotments [4, 4, 8], and boost interval 100.

Interval	CPU	Reason
0–8	A and B alternate every tick	Each gets four Q0 turns. A demotes at 7; B demotes at 8.
8–12	A	A uses its four-tick Q1 allotment, then moves to Q2 with two service ticks left.
12–16	B	B uses its Q1 allotment, then joins Q2 with two service ticks left.
16–18	A	A finishes in Q2.
18–20	B	B finishes in Q2.

Per-tick CPU sequence: A B A B A B A B A A A B B B B A A B B. No boost occurs before completion.

Process	Response	Waiting	Turnaround
A	0	8	18
B	1	10	20

Metrics and display

- **Response time:** first CPU start minus arrival. It is recorded once; preemption, demotion, and boosts do not reset it.
- **Waiting time:** cumulative ticks ready but not running; excludes time before arrival. At completion it equals turnaround minus original service. No scheduling event resets this history.
- **Turnaround time:** completion minus arrival; unavailable until completion.
- **Averages:** waiting includes all listed processes, including future processes with zero waiting. Response includes only processes that have started; turnaround includes only finished processes. Unavailable values are dashes; non-integer averages use one decimal place.
- **Budgets:** cards use Q: used/total and A: used/total. The CPU panel and metrics table show the budgets remaining. Neither counter measures total remaining service. Finished processes leave the CPU/queues, remain in the metrics table, and show dashes for remaining budgets.
- **Timeline:** a cell at t identifies the CPU during $[t, t + 1)$. A BOOST marker at t precedes that tick's execution. Playback and animation visualize recorded decisions without changing the schedule.

Input limits and recovery

Use 1–50 processes with nonempty IDs unique after trimming and ignoring case. Arrival times must be nonnegative safe whole numbers. Service, quantum, allotment, and the UI's boost interval must be positive safe whole numbers. Each queue has its own independent quantum and allotment; either may be larger, and values need not increase from Q0 to Q2.

The complete workload, including idle gaps, must finish by tick 2,000. Exactly 2,000 is accepted; 2,001 is rejected. The numeric editor floors fractions and clamps entries to the field minimum. Imported process values and direct engine inputs must pass integer validation. Invalid edits show a validation message; a rejected JSON import leaves the current scenario unchanged. Editing a valid scenario or changing the policy resets playback to zero.

Optional engine behavior outside the normal UI

The engine's optional `relinquishEarly` flag yields one tick before the current quantum expires, when that quantum is at least two ticks. It resets quantum usage, retains allotment usage, and appends to the same queue. It cannot avoid demotion by repeatedly yielding. The normal UI does not enable this flag.

Example: Quantum 3 and allotment 4 permit a two-tick early yield, then another two ticks before allotment exhaustion causes demotion. Completion and budget exhaustion take precedence over an early yield.

Engine callers may use any nonempty number of queues, with matching quantum/allotment counts. Omitting allotments uses the quanta as allotments; omitting the boost interval disables boosting. The normal UI always uses three queues and a positive boost interval, and its JSON importer does not retain the early-yield flag.

Every movement is emitted as a typed transition, including CPU-to-Q0 boosts and same-queue rotations. Forward and reverse playback use those authoritative intermediate states.